

AURA Unified Intelligence Platform

Master Product Requirements + Functional Requirements + Technical Requirements
PRD + FRD + TRD

Document	Value
Version	1.0
System	AURA Unified Intelligence Platform
Architecture	Central Brain -> Primary Agents -> Sub-agents -> Governance -> Tools -> Execution
Primary Agents	FORGE, SENTINEL, RESEARCH, AUTOMATION/OPS
Targets	Mobile, Desktop, Cloud, Local/Hybrid
Status	Master Architecture and Implementation Specification

AURA is the central brain. FORGE engineers. SENTINEL secures and verifies. RESEARCH gathers and validates knowledge. AUTOMATION/OPS performs authorized actions. Governance controls authority.

1. Executive Summary

AURA is a central AI intelligence and orchestration platform. The user primarily communicates with AURA rather than managing multiple independent agents. AURA understands objectives, maintains context, creates Missions, decomposes work, delegates tasks, coordinates agents, evaluates results, and synthesizes a unified response.

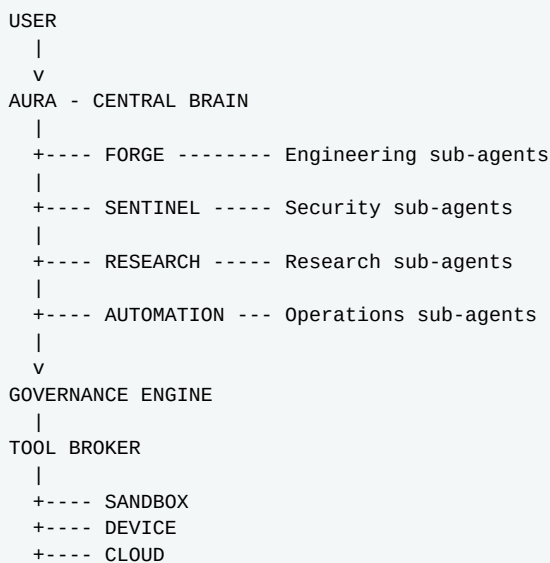
The platform contains four primary specialist systems: **FORGE** for software engineering, **SENTINEL** for defensive security and independent verification, **RESEARCH** for knowledge acquisition and evidence synthesis, and **AUTOMATION/OPS** for authorized device, infrastructure, deployment, and operational actions.

All consequential tool usage passes through an independent Governance Engine and Tool Broker. This separates AI intelligence from execution authority.

2. Product Vision - PRD

AURA should allow the user to express an objective instead of manually specifying every technical step. A command such as *"Build a full-stack AI assistant using FORGE, validate its security using SENTINEL, research anything required, test it, fix approved findings, and ask before deployment"* becomes a structured Mission with dependencies, agent assignments, policies, checkpoints, and approval gates.

3. Core Architecture



4. Product Principles

PR-01 - One central brain: AURA owns high-level intelligence and orchestration.

PR-02 - Specialized agents: Primary agents own domain-specific work.

PR-03 - Specialized sub-agents: Complex work is decomposed into narrow workers.

PR-04 - Least privilege: Agents receive only required permissions.

PR-05 - Intelligence is not authority: AI decisions do not automatically authorize execution.

PR-06 - Independent verification: FORGE cannot verify its own security fixes.

PR-07 - Human authority: Consequential actions remain policy and approval controlled.

PR-08 - Recoverability: Engineering operations should be reversible where technically possible.

PR-09 - Auditability: Consequential operations must be attributable and traceable.

PR-10 - Local-first/hybrid: Local resources should be usable alongside controlled cloud execution.

5. Product Scope and MVP

The first production-capable release focuses on AURA Core, FORGE, SENTINEL, Governance, Tool Broker, Sandbox, Git protection, Mission Control, basic Memory, and Audit. RESEARCH begins with controlled external retrieval. AUTOMATION/OPS begins heavily restricted. Broad autonomous device and cloud authority should not be introduced before governance and audit controls are mature.

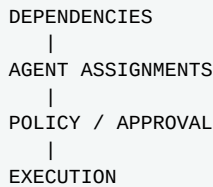
6. AURA Central Brain - Functional Requirements

ID	Requirement
FR-AURA-001	Accept normalized multimodal/structured user requests.
FR-AURA-002	Recognize one or more intents.
FR-AURA-003	Decompose objectives into goals, milestones, tasks and subtasks.
FR-AURA-004	Create dependency-aware Mission plans/DAGs.
FR-AURA-005	Route tasks by capability, policy and availability.
FR-AURA-006	Maintain scoped hierarchical context.
FR-AURA-007	Persist Mission state and artifacts.
FR-AURA-008	Support bounded dynamic replanning.
FR-AURA-009	Synthesize agent outputs into one user-facing result.
FR-AURA-010	Create and resolve approval requests.
FR-AURA-011	Recover interrupted Missions safely.
FR-AURA-012	Pause and resume Missions.
FR-AURA-013	Cancel tasks, agents or Missions.
FR-AURA-014	Monitor agent health and progress.
FR-AURA-015	Route work across configured AI models.

Core implementation components: AuraCore, IntentService, PlannerService, MissionService, TaskScheduler, AgentRouter, ContextManager, ModelRouter, ApprovalService, ResultSynthesizer and RecoveryManager.

7. Mission and Planning System

```
OBJECTIVE
|
v
GOALS
|
MILESTONES
|
TASKS
|
```



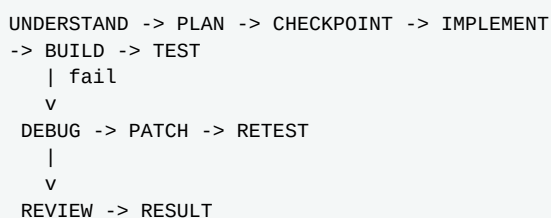
Complex objectives become Missions. Mission states include CREATED, PLANNING, WAITING_APPROVAL, RUNNING, PAUSED, BLOCKED, FAILED, COMPLETED, CANCELLED, ROLLING_BACK and ROLLED_BACK. Plans must be validated for agent/tool availability, permissions, dependency correctness, resource limits and cycles before execution.

8. Agent Architecture and Registry

Agents implement a common contract containing identity, capabilities, execute, cancel and health operations. AURA routes work by capability rather than hardcoding every workflow. Long-running agents report heartbeats and structured progress. Sub-agents receive scoped context, tools, permissions and resource limits and should normally be disposable.

9. FORGE - Engineering System

ID	Requirement
FR-FORGE-001	Inspect authorized repositories and development environment.
FR-FORGE-002	Generate architecture and implementation plans.
FR-FORGE-003	Create files within authorized workspaces.
FR-FORGE-004	Modify existing source code with Git protection.
FR-FORGE-005	Manage dependencies through policy-controlled operations.
FR-FORGE-006	Execute builds inside approved environments.
FR-FORGE-007	Create and execute unit tests.
FR-FORGE-008	Execute integration and relevant end-to-end tests.
FR-FORGE-009	Analyze failures and debug.
FR-FORGE-010	Generate bounded patches and retest.
FR-FORGE-011	Review code before engineering completion.
FR-FORGE-012	Create Git checkpoints/session branches.
FR-FORGE-013	Generate user-reviewable diffs.
FR-FORGE-014	Support rollback where technically possible.
FR-FORGE-015	Produce build/release artifacts.



10. SENTINEL - Security System

ID	Requirement
FR-SEN-001	Generate threat models for authorized applications.
FR-SEN-002	Perform static defensive source analysis.
FR-SEN-003	Analyze dependencies and known security concerns.
FR-SEN-004	Detect and redact exposed secrets.
FR-SEN-005	Review authentication controls.
FR-SEN-006	Review authorization controls.
FR-SEN-007	Review API security.
FR-SEN-008	Review security-sensitive configuration.
FR-SEN-009	Normalize findings into a common schema.
FR-SEN-010	Classify severity and confidence.
FR-SEN-011	Provide remediation recommendations.
FR-SEN-012	Independently verify FORGE remediations.
FR-SEN-013	Validate project security baselines.
FR-SEN-014	Generate auditable security reports.

Security findings use a normalized schema with finding ID, Mission, severity, category, component/location, description/evidence, confidence, recommendation, status and verification state. Secret values must never be stored in clear text in findings.

11. FORGE-SENTINEL Remediation Protocol

```

FORGE -> engineering result
|
v
AURA -> security review task
|
v
SENTINEL -> finding SEC-xxx
|
v
AURA -> remediation task
|
v
FORGE -> patch + tests
|
v
AURA -> verification task
|
v
SENTINEL -> VERIFIED / REOPEN

```

FORGE cannot change a SENTINEL finding from OPEN to VERIFIED. Verification is an independent SENTINEL operation. Accepted risk must remain distinct from resolved risk.

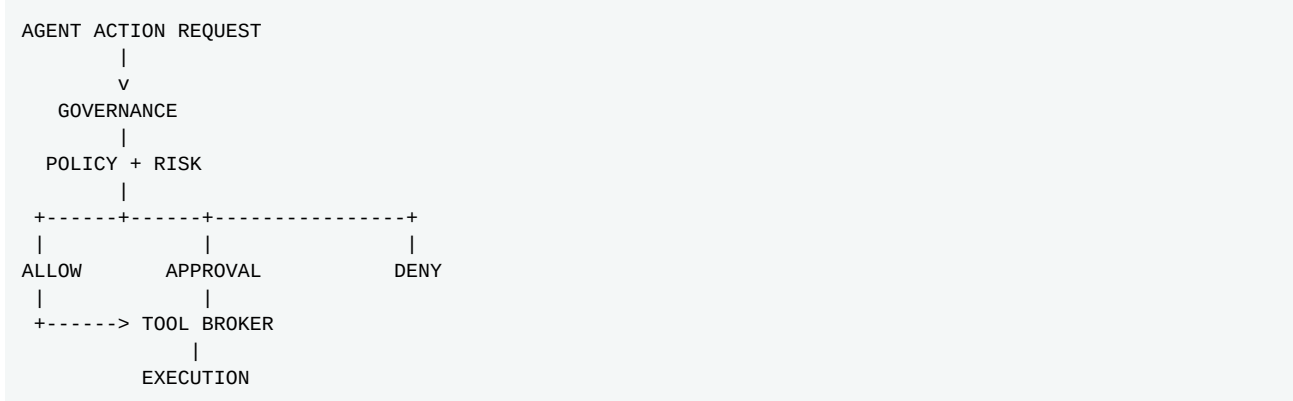
12. RESEARCH Agent

RESEARCH owns web and technical research, documentation retrieval, repository research, document analysis, fact verification, and evidence synthesis. External content is treated as untrusted data. Retrieved webpages, repositories, documents, messages, and API responses must never automatically become executable instructions.

13. AUTOMATION / OPS

AUTOMATION/OPS performs authorized external actions including device operations, file workflows, application control, cloud operations, deployment, service management, monitoring, and scheduled workflows. It should receive the most restrictive default permissions because it can create real-world side effects.

14. Governance Engine



Governance decisions: ALLOW, DENY, REQUIRE_APPROVAL, SANDBOX_ONLY and ALLOW_WITH_LIMITS. Capability permissions should be granular, such as filesystem.read, filesystem.workspace.write, git.commit, sandbox.execute, network.http.allowlisted, secrets.request, deployment.staging and deployment.production.

15. Tool Broker and Execution Layer

Agents never receive unrestricted direct tool access. Every ToolRequest includes Mission ID, Task ID, Agent ID, tool, capability, parameters, workspace and timeout. The Tool Broker obtains a Governance decision before invoking the target execution node.

16. Sandbox Requirements

```
AURA SANDBOX
- scoped /workspace
- runtime + build tools
- CPU / RAM limits
- process limits
- execution timeout
- filesystem boundary
- network policy
- credential policy
```

17. Secret Broker

Agents should not freely retrieve long-lived credentials. Secret access flows through Governance and a Secret Broker, with minimum necessary scope and preferably temporary injection into an execution environment. Secrets must be redacted from ordinary logs, prompts where unnecessary, Git commits, and UI output.

18. Context and Memory

AURA uses hierarchical context: user context -> project context -> Mission context -> agent context -> task context. Only relevant information flows downward. Persistent memory is distinct from raw execution logs and should support user visibility, editing, deletion, pinning, and disabling.

19. Model Router

AURA must not depend on a single model. Model routing may consider capability, privacy, latency, cost, context size, tool support, availability, and task complexity. The architecture must support local, cloud, and specialized models.

20. Distributed Execution Nodes

AURA may eventually route workloads across phone, laptop/desktop, sandbox, and cloud nodes. Nodes advertise capabilities, trust level, resources, runtimes, tools, health, and availability. Security and privacy constraints take priority over raw performance.

21. Event and API Architecture

```
REST / RPC:
POST /api/v1/missions
GET /api/v1/missions/:id
POST /api/v1/missions/:id/pause
POST /api/v1/missions/:id/resume
POST /api/v1/missions/:id/cancel
GET /api/v1/missions/:id/tasks
GET /api/v1/projects/:id/findings
POST /api/v1/findings/:id/remediate
GET /api/v1/approvals
POST /api/v1/approvals/:id/approve
```

```
Realtime (WebSocket/SSE):
mission.progress
agent.status
tool.output
security.finding
approval.requested
```

REST/RPC manages durable resources and commands. WebSocket or Server-Sent Events carries live Mission progress, agent state, terminal/build output, security findings and approval events. An internal event bus should begin simple; Kafka-scale infrastructure is unnecessary for the MVP.

22. Persistence and Database

```
PostgreSQL core entities:
users
devices
projects
missions
tasks
task_dependencies
agents
agent_runs
artifacts
security_findings
approvals
tool_requests
tool_executions
git_checkpoints
audit_events
memories
execution_nodes
```

PostgreSQL is the recommended initial durable store. Redis may later be introduced for queues, locks, caching and ephemeral streaming state. A vector store should only be introduced where semantic retrieval provides measurable value.

23. Git Transaction and Rollback

FORGE engineering sessions should record repository, starting HEAD, Mission branch/worktree, changes, commands, tests, security findings, artifacts, and final HEAD. Supported operations include diff review, file restore, task rollback, Mission rollback, and merge. AURA must never claim an action is reversible when the underlying side effect cannot actually be undone.

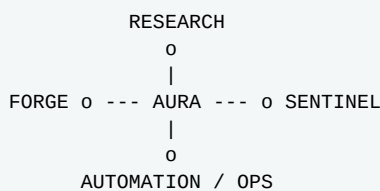
24. Failure Handling and Recovery

Autonomous retries must be bounded. A typical policy is normal execution, retry with failure context, alternative strategy, escalation to AURA, then user review. After a crash, AURA reloads Mission state, checks execution nodes, Git state, active operations, and decides whether resumption is safe. External side effects must never be blindly replayed.

25. Human-in-the-Loop and Approvals

AURA may support supervised, balanced, and autonomous modes. Autonomous mode never disables Governance. Approval previews must explain requesting agent, action, target, reason, risk, affected resources, rollback availability, and relevant SENTINEL status. Approval should be scoped to an action, task, Mission, tool, workspace, or time window.

26. UI / Mission Control Requirements



Core states:
IDLE / LISTENING / UNDERSTANDING / PLANNING
DELEGATING / ORCHESTRATING / WAITING
SPEAKING / WARNING / ERROR

Persistent controls:
PAUSE / STOP / SAFE MODE

Contextual:
ROLLBACK

The UI must show real system state rather than fake telemetry. Mission Control displays objective, progress, task dependencies, active agents/sub-agents, approvals, security findings, build/test status, Git changes, resource usage and activity history.

27. Safe Mode

A persistent AURA Safe Mode must stop new tasks, prevent new agent spawning, cancel or isolate active tool operations where possible, revoke temporary credentials, disable external actions, preserve logs and checkpoints, preserve recoverable Mission state, and keep the user interface available.

28. Security Architecture

Trust boundaries include the user, AURA Core, agents, third-party models, tools, generated code, external content, network, cloud services, and execution hosts. Data may be classified as PUBLIC, INTERNAL, CONFIDENTIAL, or

SECRET, with classification influencing model selection, cloud usage, logging, memory, agent access, and network access.

29. Observability

AURA should record Mission duration, task and agent latency, model latency, tool execution time, token usage, cost where available, CPU, RAM, storage, failures, retries, security findings, queue depth, and sandbox health. Telemetry must not expose secrets.

30. Flagship End-to-End Workflow



31. Development Roadmap

Phase	Scope	Success condition
1 - Foundation	AuraCore, MissionService, Planner, Agent Registry/Router, Content Engine, Governance, Audit	AURA Tool Broker Governance, Audit, and Able to run Missions.
2 - FORGE	Repository inspection, Git checkpoints, file operations, sandbox, UI/UX, access, file modify	AURA can safely modify and test a real repository.
3 - SENTINEL	Threat model, SAST adapters, dependency/secret analysis, no FORGE findings	FORGE -> SENTINEL Verification FORGE -> SENTINEL works end-to-end.
4 - RESEARCH	Retrieval, documentation, provenance, external-content isolation	Research can safely feed evidence into Missions.
5 - OPS	Execution nodes, deployment, monitoring, scheduled workflows	Governed real-world operations work reliably.

32. Recommended Repository Structure

```
aura/
|-- apps/
|   |-- web/
```

```

| |-- mobile/
| |-- desktop/
|-- core/
| |-- intent/
| |-- planner/
| |-- missions/
| |-- context/
| |-- routing/
| |-- models/
| |-- recovery/
|-- agents/
| |-- forge/
| |-- sentinel/
| |-- research/
| |-- automation/
|-- governance/
|-- tools/
|-- sandbox/
|-- memory/
|-- audit/
|-- database/
|-- shared/
|-- tests/

```

Start as a modular monolith. Splitting the MVP into many microservices would increase operational complexity without proving the core orchestration model.

33. Requirements Traceability

Product requirement	Functional mapping	Technical implementation
AURA orchestrates	FR-AURA-003/004/005	PlannerService + AgentRouter + MissionService
FORGE builds	FR-FORGE-*	ForgeCoordinator + workers + Tool Broker
SENTINEL verifies	FR-SEN-*	SentinelCoordinator + scanner adapters + FindingService
Safe execution	Execution requirements	SandboxManager + execution nodes
Permissions	Governance requirements	PolicyEngine + RiskEngine + ApprovalService
Rollback	FR-FORGE-012/014	GitManager + checkpoints/worktrees
Auditability	Audit requirements	AuditService + append-oriented audit_events
Live UI	Mission/UI requirements	Event bus + WebSocket/SSE

34. Definition of Done

AURA v1 is functionally proven when one high-level user objective can become a Mission; AURA plans and routes it; FORGE safely modifies and tests a real project; SENTINEL independently analyzes and revalidates security; all consequential actions pass Governance; changes are auditable and recoverable; the user can review diffs and security state; and deployment cannot occur without required authorization.